

mb-free__transport-eager-2048

An AgentMPI run analysis

Abstract

This is the last eager cell that completes. In a ten-cell transport sweep — two modes crossed with five payload sizes, all at $p = 8$ and all agent-free — eight ranks form a ring and each sends one 2,048-word payload to its successor with the mode pinned to `Mode.EAGER`. It carried all eight messages in 34 events and 0.04 s, and admitted 2,695 tokens into every receiver’s context: 8.2% of the 32,768-token budget, but 65.8% of the 4,096-token unexpected-message credit that actually gates the eager path. The gap between those two percentages is the whole point of the cell. Read against the context budget, eager looks comfortable here; read against the credit that governs admission, it has two-thirds of its headroom gone and 1,401 tokens left. The next size up, 8,192 words at 10,779 tokens, is refused outright. This is the last measurement on the safe side of a cliff whose position the run itself gives no warning about.

1 What this run is

property	value
run	mb-free__transport-eager-2048
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	<code>none</code> $\times$ 8
events	34
wall time	0.03 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	8	8 eager, 0 rendezvous
tokens sent	21,560	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	\$0.0000	0.0000 \$/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

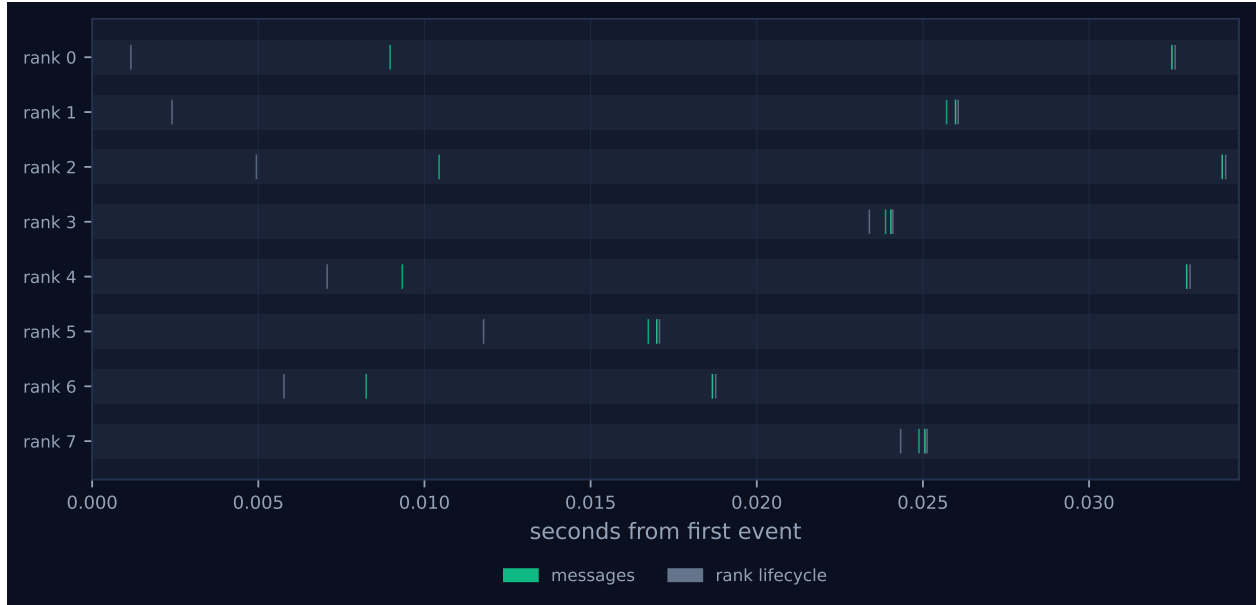


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

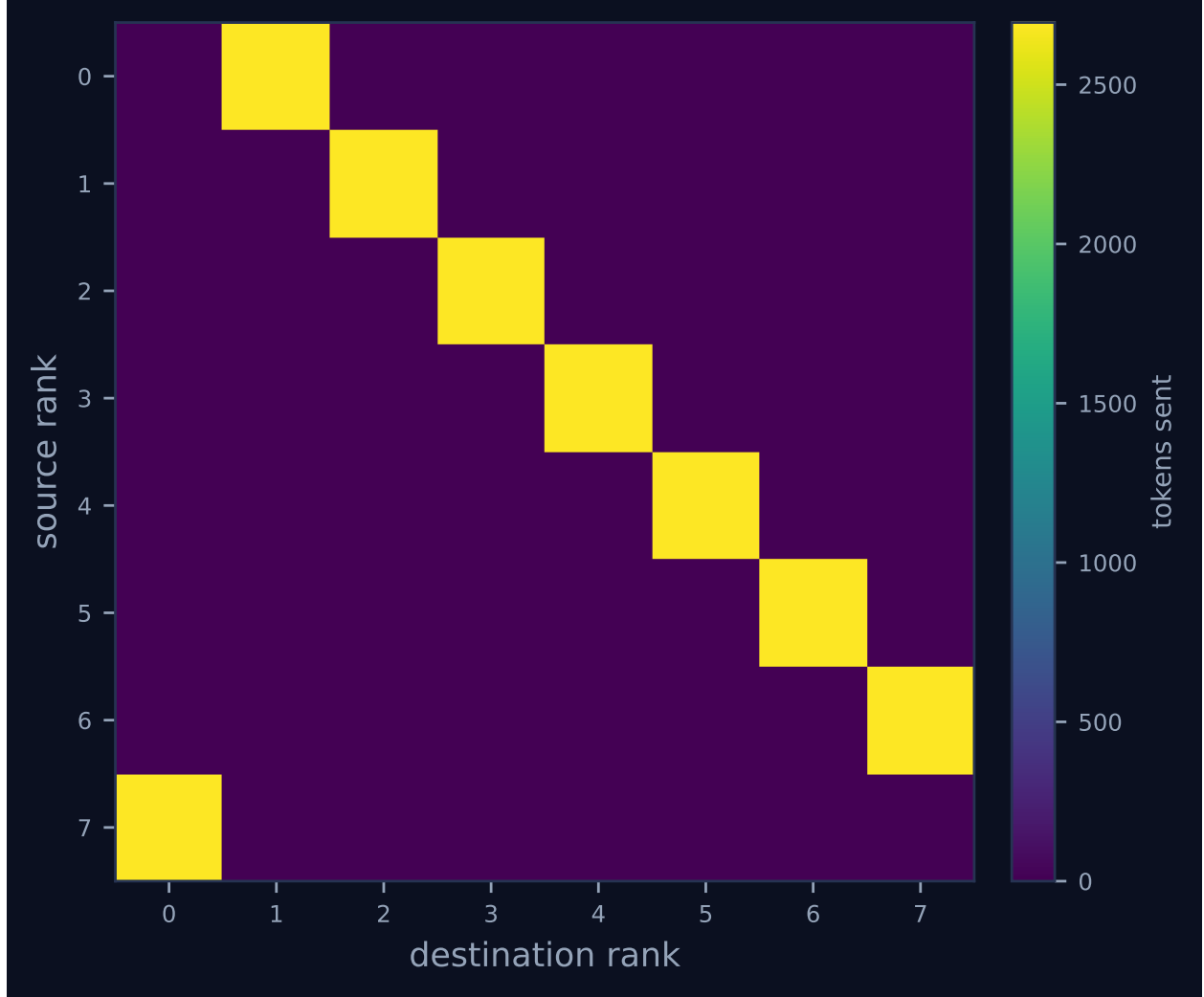


Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	1	1	2,695	8.2	finalized
1	0.00	0.0	0	0	1	1	2,695	8.2	finalized
2	0.00	0.0	0	0	1	1	2,695	8.2	finalized
3	0.00	0.0	0	0	1	1	2,695	8.2	finalized
4	0.00	0.0	0	0	1	1	2,695	8.2	finalized
5	0.00	0.0	0	0	1	1	2,695	8.2	finalized
6	0.00	0.0	0	0	1	1	2,695	8.2	finalized
7	0.00	0.0	0	0	1	1	2,695	8.2	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 shows eight lanes of instantaneous marks and no bars. The executor is **none**, there were zero agent calls and no **task.*** events, so the “idle fraction 100.0%” and “achieved parallelism 0.00×” in the headline table mean that no rank ever held a task — they are facts about an agent-free benchmark and not about the protocol. Each lane carries a **rank.init** diamond, a **msg.send** tick, a **msg.recv** tick and a **rank.finalize** diamond. Ranks arrive over a 23 ms window inside a 35 ms span, so most of the wall clock is still eight ranks starting up against one SQLite writer.

What distinguishes this cell visually from its smaller siblings is the spread between send and receive. The median receive latency is 13 ms and the maximum is 27 ms, against 4 ms and 10 ms at 128 words. No rank is waiting on a peer for any of it. The interval a latency measures starts when the sender stamps **sent_at**, and what falls inside it are two payload-sized costs on the receiver’s side: its own outgoing 10 KiB blob write, because every rank sends before it receives, and then the **blobs.get** that materialises the arriving payload, which **_deliver** performs before it emits **msg.recv**. Both scale with the payload, and the run span scales with them, from 19 ms at 128 words to 35 ms here. Nothing blocks, though. There is no **transport.credit_stall** event anywhere in the log, because the ring gives each destination exactly one inbound sender and **_await_eager_credit** therefore always finds zero outstanding queued tokens. The rank health table reads 8% occupancy across all eight ranks — the largest context footprint anywhere in this sweep, since the two eager cells above this one admitted nothing at all. Figure 2 shows the ring as a single off-diagonal band of 2,695 tokens per edge, the last time in the eager column that a communication matrix exists to be drawn.

7 Interpretation

By construction: eight ranks, a ring, one send and one receive each, mode forced. The forcing matters here more than anywhere else in the eager column. Every `rank.init` records `eager_limit: 1000000000`, which disables `_choose_mode`’s automatic threshold. That threshold defaults to `DEFAULT_EAGER_LIMIT = 2048 tokens`, and this payload is 2,695 tokens, so under the shipped default the runtime would already have converted this send to rendezvous. This cell is thus the first in the sweep where the harness is overriding what AgentMPI would have done on its own, and it still succeeds — the default eager limit is conservative relative to the 4,096-token credit that actually enforces anything.

What emerged is how little warning the successful case gives. The generated finding list is empty; the job outcome is “completed”; every rank finalised with zero rejections and 8.2% occupancy. Nothing in this run’s own record indicates that it is close to a limit, because the limit is not the context budget it reports occupancy against. The binding resource is the destination’s `unexpected_limit`, fixed at 4,096 tokens by the benchmark, and `_await_eager_credit` checks a single payload against it before doing anything else: `if n_tokens > limit: raise AmpContextOverflow`. At 2,695 tokens this cell passes with 1,401 to spare, 34% of the credit. Its successor at 8,192 words prices at 10,779 tokens, fails that check, and its trace contains no `msg.send` at all — 18 events instead of 34, an empty `messages` table in the fabric, and `completed: false` with `ERR_CONTEXT_OVERFLOW` in `results/microbench/free-transport.json`. Since the charge is linear at $5/3.8 = 1.3158$ tokens per word with no intercept (168, 674 and 2,695 tokens for 128, 512 and 2,048 words), the crossing is at 3,113 words. This cell is at 66% of the way there and reports itself healthy. That is the argument for making transport mode explicit: a harness cannot infer from a successful eager run how much eager headroom it has left, because the successful run reports occupancy against the wrong budget.

Against its rendezvous twin, this cell is the last place in the sweep where the two modes are genuinely comparable. `mb-free__transport-rendezvous-2048` moves the identical blob in the identical 34 events and reports zero admissions; this one reports 2,695 tokens per rank. At 8.2% of budget the eager charge is affordable and the receiver gets the content, whereas the rendezvous run’s receivers finish holding a handle they never opened. Above this size that comparison stops being available, because the eager arm no longer runs. Everything the sweep says about rendezvous being necessary rather than merely available rests on the two cells above this one, and everything it says about eager being adequate rests on this one and the two below.

8 Threats to this reading

The context figures are admissions rather than usage: with `executor: none` and no agent calls, 2,695 tokens were charged to a budget nothing read, so this run measures the protocol’s accounting and not any agent’s behaviour. The token count is an estimate (`tokenizer_exact: false`) produced by a character heuristic on a payload of one repeated word; a real tokeniser would price this text nearer 2,048 tokens, which would put it at 50% rather than 66% of credit and move the sweep’s ceiling from 3,113 words to roughly 4,096. The ceiling’s existence does not depend on that, but its position in this document does, and no cell in the sweep brackets it more tightly than the factor of four between 2,048 and 8,192 words. The wall time is one sample — 35 ms, of which 23 ms is rank start-up — so the observation that latencies are larger here than at 128 words is suggestive of blob write cost but is not cleanly separable from start-up jitter at this scale. Finally, the claim that this cell is “close to the limit” is a claim about a per-message precondition, not

about buffering. The ring hands each receiver exactly one message, so the accumulation path that `_await_eager_credit` spends most of its code on — polling outstanding queued tokens, emitting `transport.credit_stall`, blocking until credit frees — is never entered in any of these ten runs. A harness that fanned two of these 2,695-token messages into one rank would exceed the same 4,096-token credit by accumulation and would find out through a stall rather than a refusal, and that experiment is absent here.